

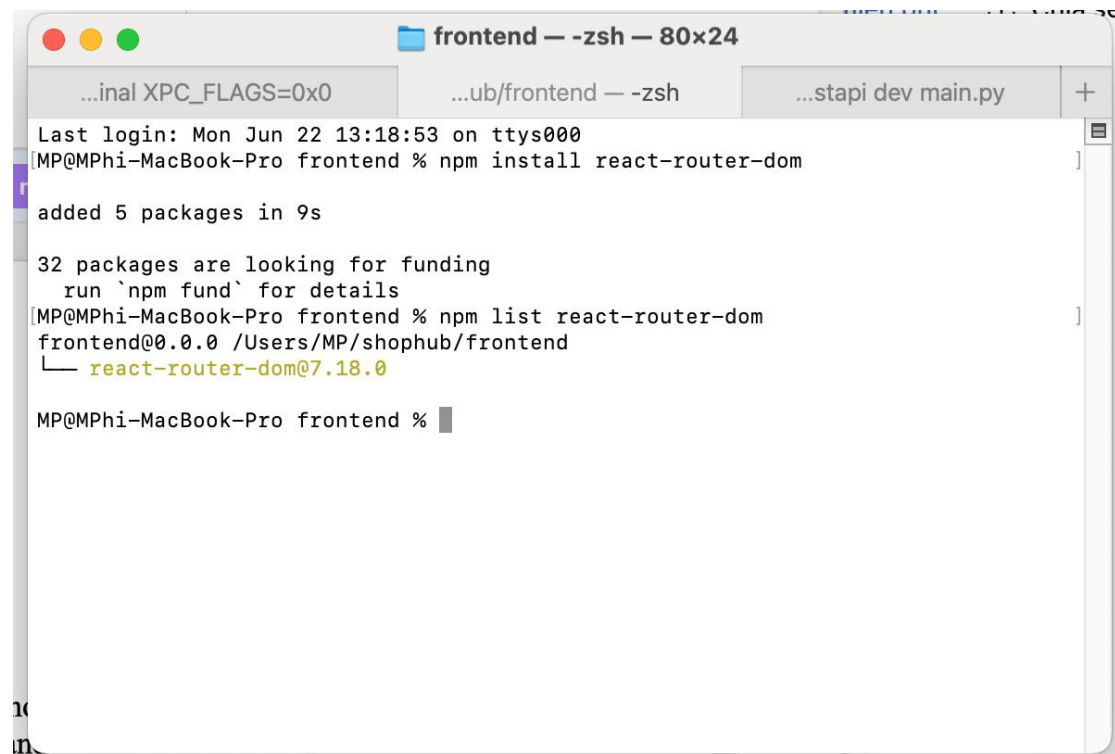
SESSION 05 – MULTI-PAGE APPLICATION WITH REACT

1. Objective

The objective of Session 05 is to transform ShopHub from a single-page product catalog into a Single Page Application (SPA) using React Router. Students learn how to create multiple pages, navigate between pages without reloading the browser, and display dynamic content based on URL parameters.

2. Technologies Used

- React
- React Router DOM
- Axios
- JavaScript (ES6+)
- CSS

A screenshot of a macOS terminal window titled 'frontend — -zsh — 80x24'. The terminal shows the execution of 'npm install react-router-dom' and 'npm list react-router-dom'. The output indicates that 5 packages were added in 9 seconds and that 32 packages are looking for funding. The 'npm list' command shows 'react-router-dom@7.18.0' as a dependency. The terminal prompt is 'MP@MPHi-MacBook-Pro frontend %'.`frontend — -zsh — 80x24
...inal XPC_FLAGS=0x0 ...ub/frontend — -zsh ...stapi dev main.py +
Last login: Mon Jun 22 13:18:53 on ttys000
MP@MPHi-MacBook-Pro frontend % npm install react-router-dom
added 5 packages in 9s
32 packages are looking for funding
 run `npm fund` for details
MP@MPHi-MacBook-Pro frontend % npm list react-router-dom
frontend@0.0.0 /Users/MP/shophub/frontend
└─ react-router-dom@7.18.0
MP@MPHi-MacBook-Pro frontend %`

3. Main Concepts

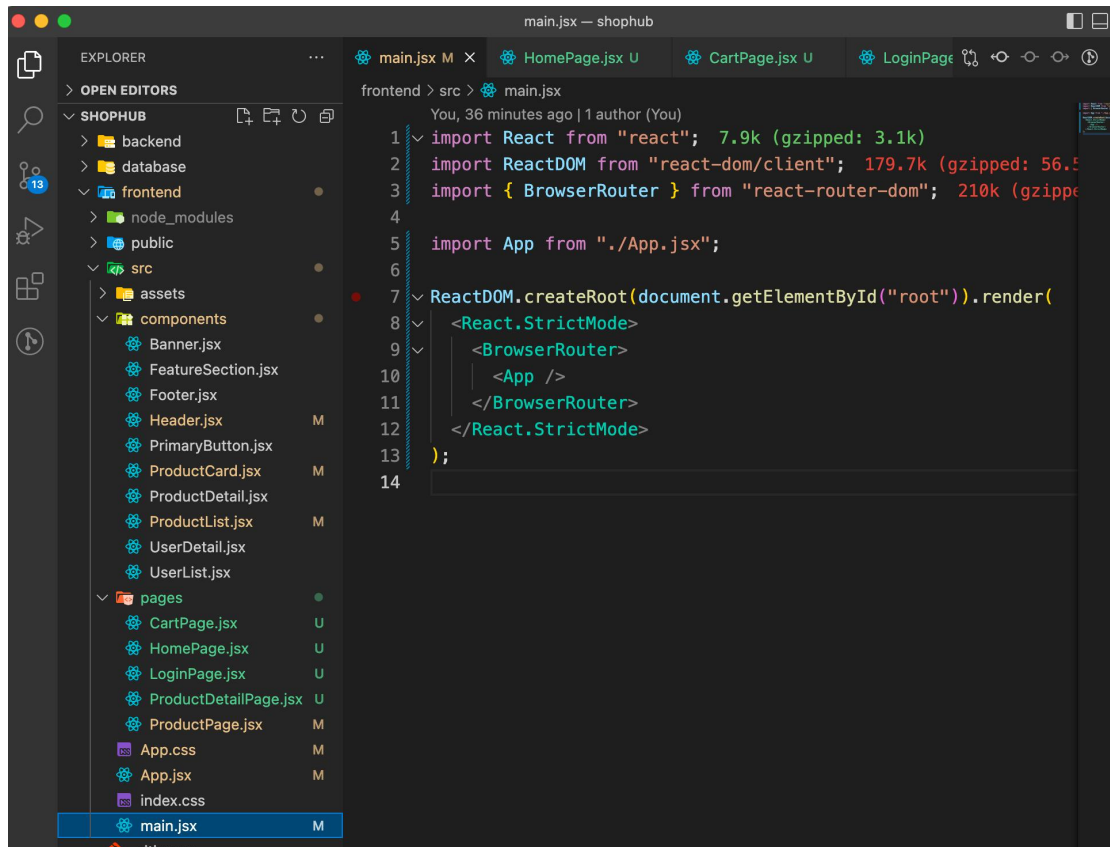
3.1 Single Page Application (SPA)

A Single Page Application loads only one HTML page and dynamically updates the content without reloading the browser. React Router is used to manage navigation between pages.

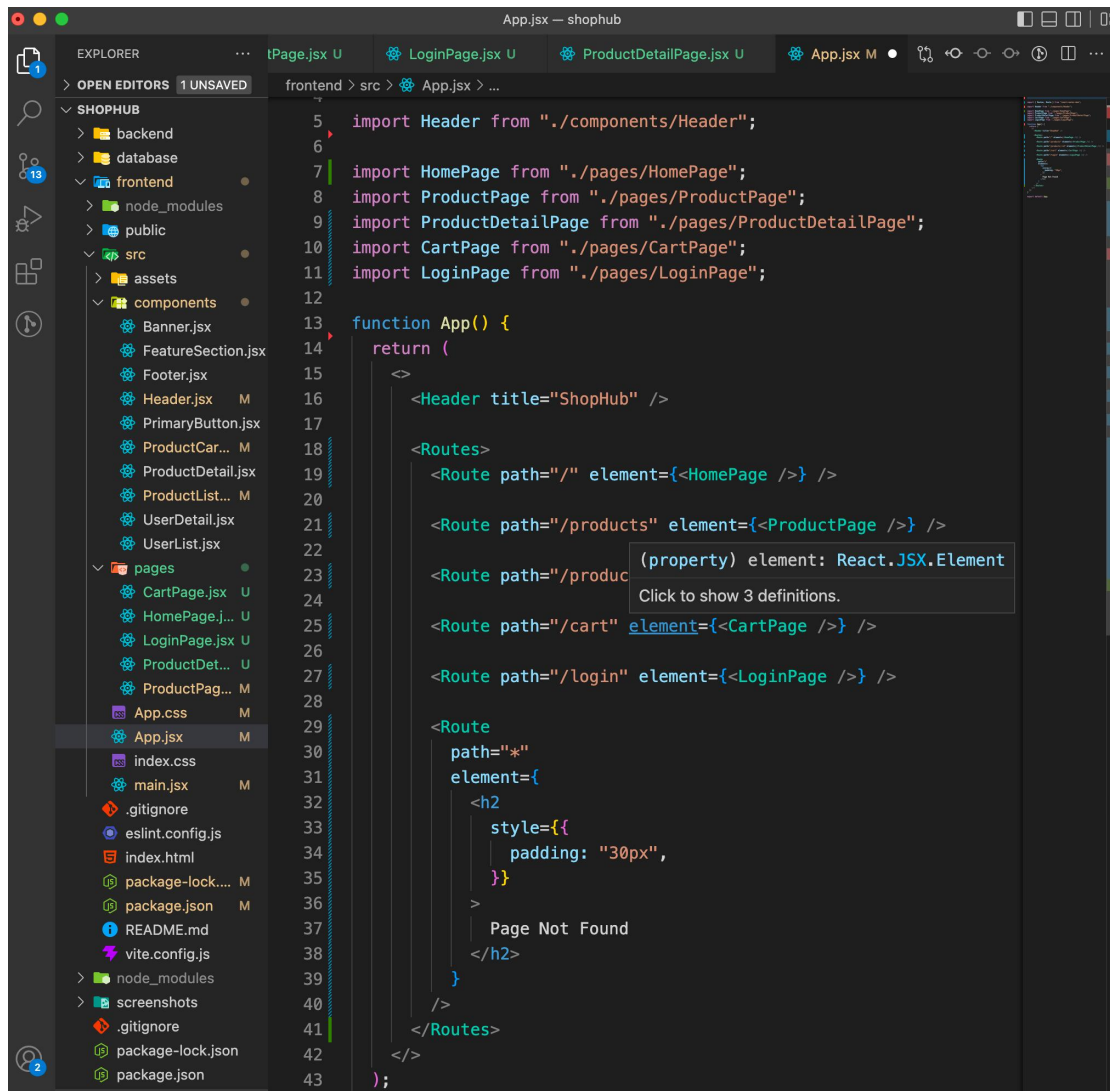
3.2 React Router

The following React Router features were implemented:

- BrowserRouter



- Routes
- Route

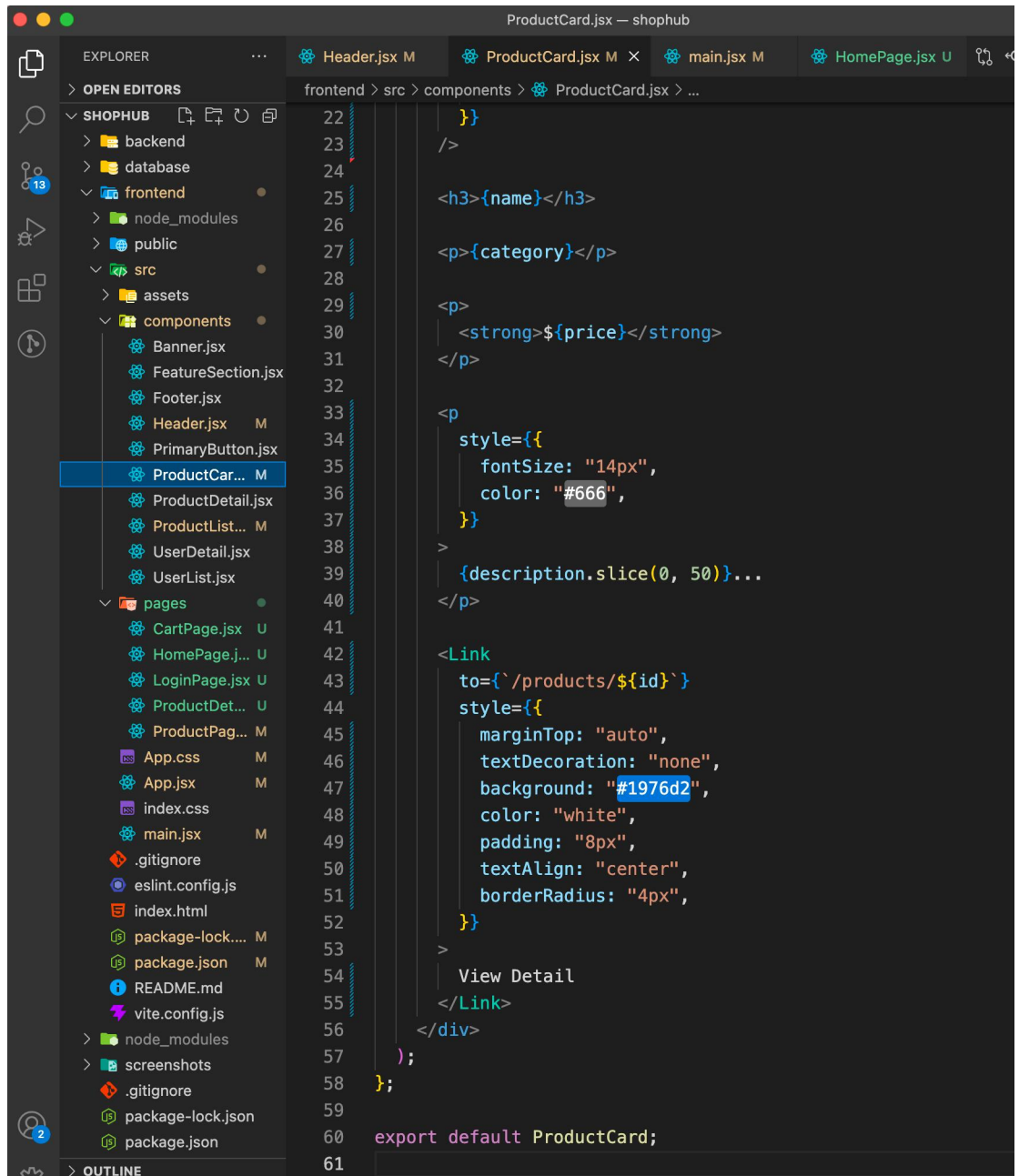


- NavLink

The screenshot shows a VS Code editor window with the title 'Header.jsx - shophub'. The Explorer sidebar on the left shows the project structure: SHOPHUB (backend, database, frontend, public, src, assets, components, pages), App.css, App.jsx, index.css, main.jsx, .gitignore, eslint.config.js, index.html, package-lock.json, package.json, README.md, vite.config.js, node_modules, screenshots, .gitignore, package-lock.json, and package.json. The 'components' folder is expanded, and 'Header.jsx' is selected. The main editor area shows the code for Header.jsx, which defines a Header component with a title and a navigation bar. The code is as follows:

```
8
9
10
11 { label: "Cart", to: "/cart" },
12
13 { label: "Login", to: "/login" },
14
15 };
16
17 const linkStyle = ({ isActive }) => ({
18   marginLeft: "15px",
19
20   textDecoration: "none",
21
22   color: isActive ? "#1976d2" : "#555",
23
24   fontWeight: isActive ? "bold" : "normal",
25 });
26
27 return (
28   <header
29     style={{
30       display: "flex",
31       justifyContent: "space-between",
32       alignItems: "center",
33       padding: "20px 24px",
34       borderBottom: "1px solid #ddd",
35     }}
36   >
37     <h1 style={{ margin: 0 }}>{title}</h1>
38
39     <nav>
40       {navItems.map((item) => (
41         <NavLink key={item.to} to={item.to} style={linkStyle}>
42           {item.label}
43         </NavLink>
44       ))}
45     </nav>
46   </header>
47 );
48
49 export default Header;
```

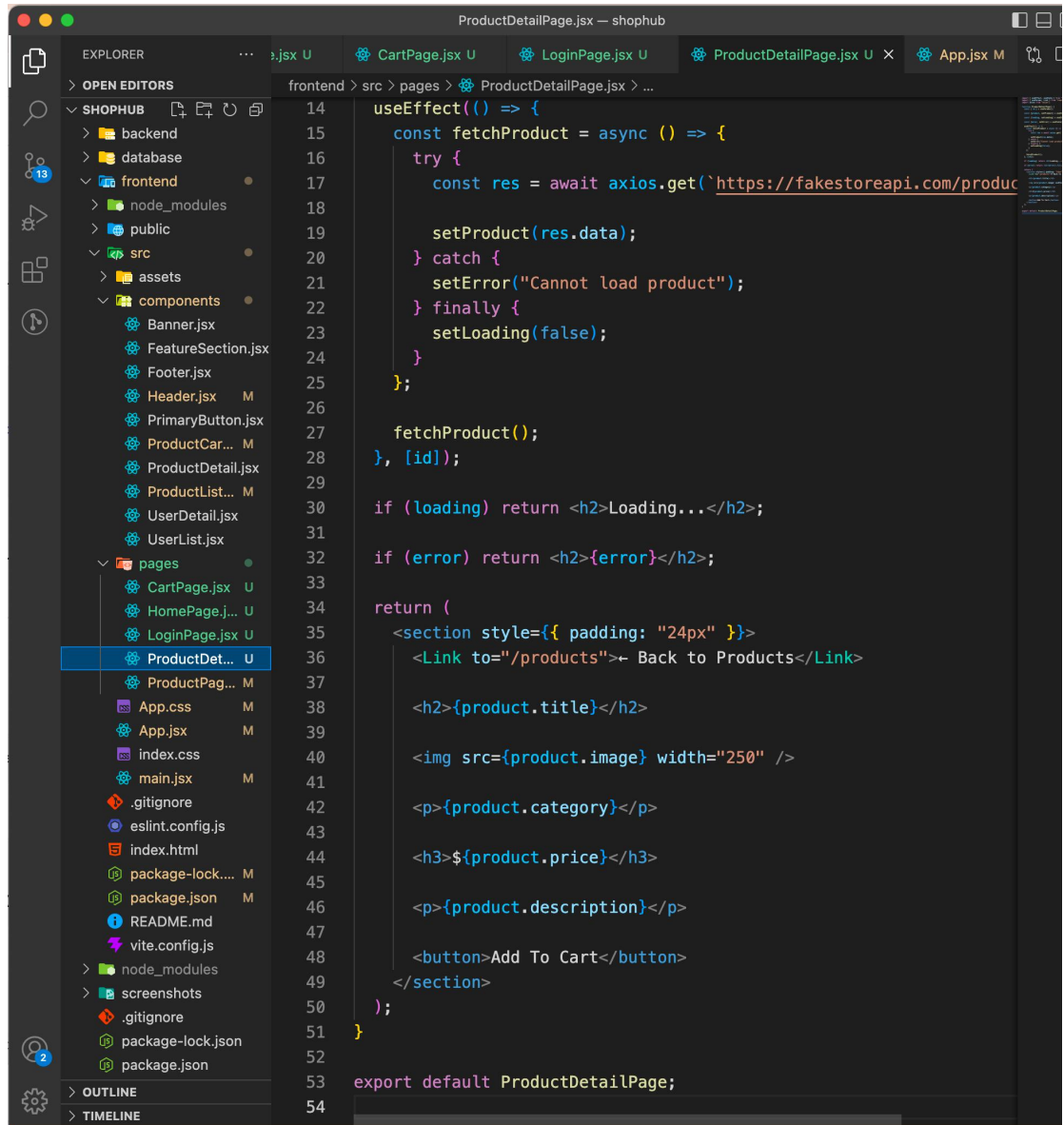
- Link



The screenshot shows a VS Code editor window with the title 'ProductCard.jsx — shophub'. The Explorer sidebar on the left displays the project structure, with the 'components' folder expanded and 'ProductCard.jsx' selected. The main editor area shows the code for 'ProductCard.jsx', which includes JSX elements for a product card and a 'View Detail' link. The code is as follows:

```
22 |   }  
23 | />  
24 |  
25 |   <h3>{name}</h3>  
26 |  
27 |   <p>{category}</p>  
28 |  
29 |   <p>  
30 |     <strong>${price}</strong>  
31 |   </p>  
32 |  
33 |   <p  
34 |     style={{  
35 |       fontSize: "14px",  
36 |       color: "#666",  
37 |     }}  
38 |   >  
39 |     {description.slice(0, 50)}...  
40 |   </p>  
41 |  
42 |   <Link  
43 |     to={` /products/${id}`}  
44 |     style={{  
45 |       marginTop: "auto",  
46 |       textDecoration: "none",  
47 |       background: "#1976d2",  
48 |       color: "white",  
49 |       padding: "8px",  
50 |       textAlign: "center",  
51 |       borderRadius: "4px",  
52 |     }}  
53 |   >  
54 |     View Detail  
55 |   </Link>  
56 | </div>  
57 | );  
58 | };  
59 |  
60 | export default ProductCard;  
61 |
```

- useParams



These features allow navigation between pages and support dynamic URLs.

4. Implementation

4.1 Routing Configuration

The application was configured with the following routes:

- Home Page (/)
- Products Page (/products)
- Product Detail Page (/products/)
- Cart Page (/cart)
- Login Page (/login)

- Not Found Page (*)

4.2 Navigation Bar

The Header component was updated to use NavLink for navigation. Active routes are highlighted and page transitions occur without browser refresh.

4.3 Home Page

The Home page contains:

- ShopHub banner
- Welcome message
- Introduction section
- Shop Now button

4.4 Products Page

The Products page reuses the functionality from Session 04:

- Fetch product data from FakeStore API
- Search products by name
- Filter products by category
- Sort products by price
- Clear filters
- Display product count

4.5 Product Detail Page

The Product Detail page uses useParams() to read the product ID from the URL.

Features:

- Display product image
- Display product title
- Display category
- Display price
- Display description
- Back to Products link
- Add To Cart button (UI only)

4.6 Cart Page

A placeholder Shopping Cart page was created containing:

- Sample cart information
- Checkout button
- Continue Shopping button

4.7 Login Page

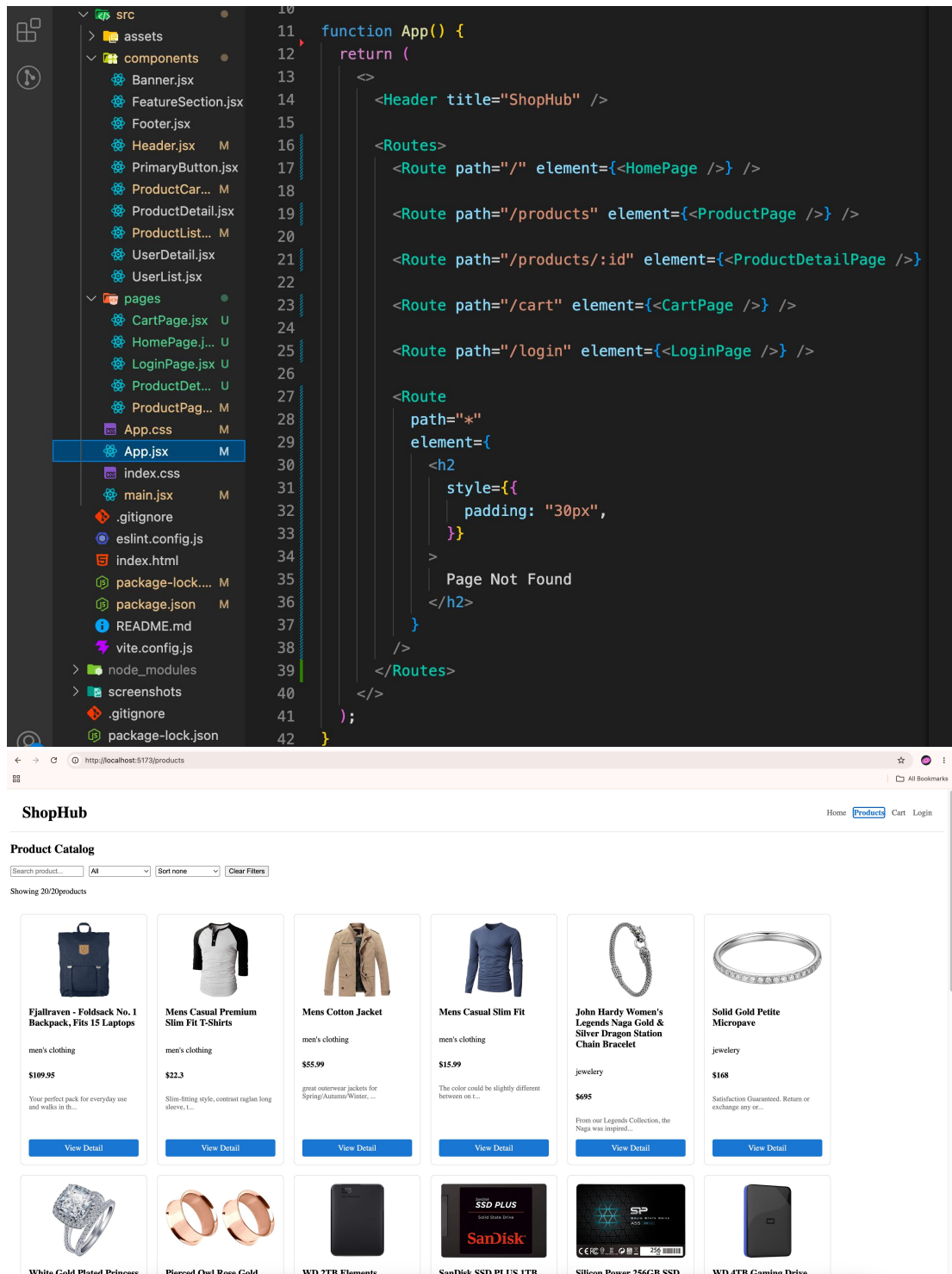
A simple login form was created with:

- Email/Username field
- Password field
- Login button

No authentication logic was implemented in this session.

4.8 Not Found Page

A 404 page was added to handle invalid URLs and display a Page Not Found message.



5. Results

After completing Session 05, ShopHub supports:

- Multi-page navigation
- Dynamic product detail pages

- React Router integration
- SPA behavior without page reloads
- Better project structure with reusable pages and components

